

Data Structures - Lab

Content

BST Fundamentals

- Insertion of nodes into a BST without duplicates.
- Traversals (Inorder, Preorder, Postorder) with recursive implementation.
- Searching for a key in BST and reporting presence/absence.

Advanced BST Operations

- Recursive deletion of a node in BST: handling cases of leaf nodes, single child, and two children.
- Recursive function to check whether two BSTs are identical (structure + data).

Real-Life Applications of BST

- **Library Catalog System:** Convert a BST (ISBN-based storage) into a sorted singly linked list for cataloging.
- **Citizen ID / Database Scenarios** (extension idea): Perform range queries, analytics, or efficient record lookups using BST properties.

Recursive Thinking with BST

- Applying recursion for insertion, traversal, and transformations.
- Understanding backtracking and recursive return values in tree problems.

Task 1

Imagine you are working as a junior software developer at a company that manages large amounts of numeric data (such as student IDs, product codes, or transaction numbers). The company has asked you to design a system using a **Binary Search Tree (BST)** to organize this data efficiently.

Your task is to **implement a C++ program** that performs the following operations:

1. Insert a Sequence of Numbers into a BST

- o Take input from the user in the form of integers (e.g., 50 30 70 20 40 60 80).
- o Insert each number into the BST following the BST property:
 - Values smaller than the current node go to the **left child**.
 - Values greater than the current node go to the **right child**.
- o Make sure that duplicate values are not inserted.

2. Display the BST Using Traversals

- o Implement three recursive functions to traverse the BST:
 - **Inorder Traversal (Left → Root → Right):** This should display the numbers in sorted order.
 - **Preorder Traversal (Root → Left → Right):** This helps visualize the structure of the tree.
 - **Postorder Traversal (Left → Right → Root):** This is often used in deletion and evaluation problems.
- o Print the output of each traversal clearly.

3. Search for a Given Key in the BST

- o Prompt the user to enter a value (key) to search for in the BST.
- o Implement a search function that recursively checks whether the key exists in the tree.
- o If the key is found, display "**Key found in BST**". Otherwise, display "**Key not found in BST**".

Your task is to implement a **recursive delete function** in C++ that removes a given key from the BST. You must handle the following **three cases**:

1. Case 1 – Leaf Node (No Children):

If the node to be deleted has no children, simply remove it.

Example: Deleting 20 from the tree 50 30 70 20 40 results in 50 30 70 40.

2. Case 2 – Node with One Child:

If the node has only one child, bypass the node and link its parent directly to its child.

Example: Deleting 30 from the tree 50 30 40 70 results in 50 40 70.

3. Case 3 – Node with Two Children:

If the node has two children, replace the node with either its **inorder successor** (smallest value in the right subtree) or **inorder predecessor** (largest value in the left subtree).

Example: Deleting 50 from the tree 50 30 70 20 40 60 80 results in 60 becoming the new root, maintaining BST properties.

Task 2

In this task, you are required to write a C++ program that checks whether **two Binary Search Trees (BSTs)** are identical or not. Two BSTs are considered **identical** if they have:

1. The **same structure** (i.e., nodes are arranged in the same way).
2. The **same data values** stored at corresponding nodes

What You Need to Do

- Implement a recursive function `bool isIdentical(Node* root1, Node* root2)` that
 - o Returns **true** if both BSTs are identical.
 - o Returns **false** if they differ in structure or data.
- Build **two BSTs** by inserting a given sequence of numbers.
- Use **inorder traversal** to display both BSTs before performing the check.
- Display a message stating whether the two BSTs are identical or not.

Task 3

Imagine a **library system** where each book is stored in a **Binary Search Tree (BST)** using its **ISBN number** as the unique key. The BST allows fast searching, insertion, and deletion of books, but when it comes to creating a **sorted catalog** of all books, a linked list is more efficient for sequential traversal.

Your task is to **convert the BST into a sorted singly linked list** using **recursion**. The conversion should maintain the natural ascending order of ISBNs, which can be achieved by performing an **inorder traversal** of the BST. Each visited node should be inserted into the linked list in the same order

Detailed Requirements:

1. **Input:** A BST where each node contains:
 - o ISBN (integer key)
 - o Title (string for book title)
 - o Left and right pointers
2. **Output:** A singly linked list where each node contains:
 - o ISBN
 - o Title
 - o Pointer to the next node
3. **Operations to Implement:**
 - o Insert books into the BST by ISBN.
 - o Perform an **inorder traversal** to process nodes in sorted order.
 - o Recursively convert the BST into a sorted singly linked list.
 - o Print the linked list to show the catalog.